

NBA DATA PROJECT ANALYSIS

player_game_data DataFrame as player_data

SELECT * FROM 'player_game_data.csv'

...	↑↓	n	...	↑↓	gamedate	...	↑↓	...	↑↓	nb...	...	↑↓	player_name	...	↑↓	n...	...	↑↓	...
0		21700826			2018-02-10T00:00:00.000			2017		2			1627821 James Webb III			1610612751			BKN
1		21700826			2018-02-10T00:00:00.000			2017		2			1626156 D'Angelo Russell			1610612751			BKN
2		21700826			2018-02-10T00:00:00.000			2017		2			203917 Nik Stauskas			1610612751			BKN
3		21700826			2018-02-10T00:00:00.000			2017		2			1626143 Jahlil Okafor			1610612751			BKN
4		21700826			2018-02-10T00:00:00.000			2017		2			202391 Jeremy Lin			1610612751			BKN
5		21700826			2018-02-10T00:00:00.000			2017		2			203915 Spencer Dinwiddie			1610612751			BKN
6		21700826			2018-02-10T00:00:00.000			2017		2			201960 DeMarre Carroll			1610612751			BKN
7		21700826			2018-02-10T00:00:00.000			2017		2			203925 Joe Harris			1610612751			BKN
8		21700826			2018-02-10T00:00:00.000			2017		2			203112 Quincy Acy			1610612751			BKN
9		21700826			2018-02-10T00:00:00.000			2017		2			1628495 Milton Doyle			1610612751			BKN
10		21700826			2018-02-10T00:00:00.000			2017		2			1627785 Isaiah Whitehead			1610612751			BKN
11		21700826			2018-02-10T00:00:00.000			2017		2			202389 Timofey Mozgov			1610612751			BKN
12		21700826			2018-02-10T00:00:00.000			2017		2			1627747 Caris LeVert			1610612751			BKN
13		21700826			2018-02-10T00:00:00.000			2017		2			1626178 Rondae Hollis-Jefferson			1610612751			BKN
14		21700826			2018-02-10T00:00:00.000			2017		2			1628386 Jarrett Allen			1610612751			BKN
15		21700826			2018-02-10T00:00:00.000			2017		2			203459 Allen Crabbe			1610612751			BKN

Rows: 1,694  Truncated from 434,797 rows Expand

team_game_data DataFrame as team_data

-- Explore the data in the table

SELECT *
FROM 'team_game_data.csv'

...	↑↓	...	↑↓	...	↑↓	n	...	↑↓	gamedate	...	↑↓	offensivenate...	...	↑↓	off_team_name	...	↑↓	...	↑↓
0		2016			2	21600495			2016-12-30T00:00:00.000			1610612740			New Orleans Pelicans			NOP	
1		2016			2	21600495			2016-12-30T00:00:00.000			1610612752			New York Knicks			NYK	
2		2021			2	22100943			2022-03-03T00:00:00.000			1610612742			Dallas Mavericks			DAL	
3		2021			2	22100943			2022-03-03T00:00:00.000			1610612744			Golden State Warriors			GSW	
4		2016			2	21601032			2017-03-18T00:00:00.000			1610612741			Chicago Bulls			CHI	
5		2016			2	21601032			2017-03-18T00:00:00.000			1610612762			Utah Jazz			UTA	
6		2021			2	22100942			2022-03-03T00:00:00.000			1610612761			Toronto Raptors			TOR	
7		2022			2	22200482			2022-12-23T00:00:00.000			1610612750			Minnesota Timberwolves			MIN	
8		2016			2	21600494			2016-12-30T00:00:00.000			1610612738			Boston Celtics			BOS	
9		2016			2	21600494			2016-12-30T00:00:00.000			1610612748			Miami Heat			MIA	
10		2022			2	22200495			2022-12-25T00:00:00.000			1610612738			Boston Celtics			BOS	
11		2022			2	22200495			2022-12-25T00:00:00.000			1610612749			Milwaukee Bucks			MIL	
12		2017			2	21700768			2018-02-02T00:00:00.000			1610612747			Los Angeles Lakers			LAL	
13		2017			2	21700768			2018-02-02T00:00:00.000			1610612751			Brooklyn Nets			BKN	
14		2023			2	22300197			2023-11-15T00:00:00.000			1610612750			Minnesota Timberwolves			MIN	
15		2016			2	21600493			2016-12-30T00:00:00.000			1610612751			Brooklyn Nets			BKN	

Rows: 2,439  Truncated from 27,144 rows Expand

```
print(player_data)
```

	nbagameid	...	opponentteamfg3attempted
0	21700826	...	4
1	21700826	...	12
2	21700826	...	5
3	21700826	...	2
4	21700826	...	0
...	...	...	...
434792	21700828	...	20
434793	21700828	...	23
434794	21700828	...	3
434795	21700828	...	28
434796	21700828	...	0

[434797 rows x 59 columns]

player_data.head()

...	↑↓	n	...	↑↓	gamedate	...	↑↓	...	↑↓	...	↑↓	nb...	...	↑↓	player_name	...	↑↓	n...	...	↑↓	...	↑↓	team_...
	0	21700826			2018-02-10T00:00:00.000			2017			2	1627821			James Webb III			1610612751			BKN		Brooklyn
	1	21700826			2018-02-10T00:00:00.000			2017			2	1626156			D'Angelo Russell			1610612751			BKN		Brooklyn
	2	21700826			2018-02-10T00:00:00.000			2017			2	203917			Nik Stauskas			1610612751			BKN		Brooklyn
	3	21700826			2018-02-10T00:00:00.000			2017			2	1626143			Jahlil Okafor			1610612751			BKN		Brooklyn
	4	21700826			2018-02-10T00:00:00.000			2017			2	202391			Jeremy Lin			1610612751			BKN		Brooklyn

Rows: 5

Expand

Question 1: What was the Warriors' Team offensive and defensive eFG% in the 2015-16 regular season? Remember that this is in the data as the 2015 season.

```

import pandas as pd

# Calculate eFG
team_data['eFG'] = (team_data['fgmade'] + 0.5 * team_data['fg3made']) / team_data['fgattempted']
team_data['eFG'] = team_data['eFG']*100

# Filter for regular season
team_data_reg_season = team_data[team_data['gametype'] == 2]

# Filter for Warriors 2015 (either offensive or defensive)
warriors_2015 = team_data_reg_season[
    (team_data_reg_season['season'] == 2015) &
    ((team_data_reg_season['off_team'] == 'GSW') | (team_data_reg_season['def_team'] == 'GSW'))
]

# Split into defensive and offensive stats
warriors_2015_def = warriors_2015[warriors_2015['def_team'] == 'GSW']
warriors_2015_off = warriors_2015[warriors_2015['off_team'] == 'GSW']

# Calculate average eFG for defensive stats
average_efg_def = warriors_2015_def.groupby('season')['eFG'].mean().round(1).reset_index()
average_efg_def.columns = ['season', 'mean_eFG']
average_efg_def['mean_eFG'] = average_efg_def['mean_eFG'].round(1).astype(str) + '%'

print("Warriors 2015 defensive eFG")
print(average_efg_def)

# Calculate average eFG for offensive stats
average_efg_off = warriors_2015_off.groupby('season')['eFG'].mean().round(1).reset_index()
average_efg_off.columns = ['season', 'mean_eFG']
average_efg_off['mean_eFG'] = average_efg_off['mean_eFG'].round(1).astype(str) + '%'
print("Warriors 2015 offensive eFG")
print(average_efg_off)

```

```

Warriors 2015 defensive eFG
  season mean_eFG
0   2015    47.9%

Warriors 2015 offensive eFG
  season mean_eFG
0   2015    56.5%

```

Question 2: What percent of the time does the team with the higher eFG% in a given game win that game? Use games from the 2014-2023 regular seasons. If the two teams have an exactly equal eFG%, remove that game from the calculation.

```

# Filter for seasons 2014-2023
team_data_14_23 = team_data_reg_season[
    (team_data_reg_season['season'] >= 2014) &
    (team_data_reg_season['season'] <= 2023)
]

# Group by game and calculate lagged differences
team_data_14_23 = team_data_14_23.sort_values('nbageid').copy()
team_data_14_23['efg_diff'] = team_data_14_23.groupby('nbageid')['eFG'].diff()
team_data_14_23['off_rebound_diff'] = team_data_14_23.groupby('nbageid')['reboffensive'].diff()

# Getting the difference between offensive and defensive rebounding by row line
# For first instance of each game, calculate difference with second row
first_instance_mask = ~team_data_14_23.duplicated(subset='nbageid', keep='first')
second_instance_mask = ~team_data_14_23.duplicated(subset='nbageid', keep='last')

# Get first and second rows for each game
first_rows = team_data_14_23[first_instance_mask].copy()
second_rows = team_data_14_23[second_instance_mask].copy()

# Calculate differences for first instances
team_data_14_23.loc[first_instance_mask, 'efg_diff'] = (
    first_rows['eFG'].values - second_rows['eFG'].values
)
team_data_14_23.loc[first_instance_mask, 'off_rebound_diff'] = (
    first_rows['reboffensive'].values - second_rows['reboffensive'].values
)

# Removing if they are the same
efg_remove_same = team_data_14_23[team_data_14_23['efg_diff'] != 0]

# Showing the proportion of if higher efg teams win
efg_games = efg_remove_same['nbageid'].nunique()
higher_efg_team = efg_remove_same[
    (efg_remove_same['off_win'] == 1) &
    (efg_remove_same['efg_diff'] > 0)
]
proportion_higher_efg_team = len(higher_efg_team) / efg_games
print("Teams that win the eFG% battle win")
print(f'{round(proportion_higher_efg_team * 100, 2)}% of the time')

```

Teams that win the eFG% battle win
81.6% of the time

Question 3: What percent of the time does the team with more offensive rebounds in a given game win that game? Use games from the 2014-2023 regular seasons. If the two teams have an exactly equal number of offensive rebounds, remove that game from the calculation.

```

# Same statistics but looking at rebounding difference
remove_off_rebound = team_data_14_23[team_data_14_23['off_rebound_diff'] != 0.0]

off_rebound_games = remove_off_rebound['nbageid'].nunique()
higher_off_rebound = remove_off_rebound[
    (remove_off_rebound['off_win'] == 1) &
    (remove_off_rebound['off_rebound_diff'] > 0)
]
proportion_higher_rebound = len(higher_off_rebound) / off_rebound_games * 100
print("Teams that get more offensive rebounds win")
print(f'{round(proportion_higher_rebound, 1)}% of the time')

```

'More offensive opportunities do not always lead to more points scored. Teams that have less opportunities because of made shots will often have lower offensive rebounds. Additionally, sometimes offensive rebounding can be turned into more missed shots, offensive fouls, turnovers, etc. Would need further analysis to be certain.'

Teams that get more offensive rebounds win
46.2% of the time

'More offensive opportunities do not always lead to more points scored. Teams that have less opportunities because of made shots will often have lower offensive rebounds. Additionally, sometimes offensive rebounding can be turned into more missed shots, offensive fouls, turnovers, etc. Would need further analysis to be certain.'

Question 4: Look at players who played at least 25% of their possible games in a season and scored at least 25 points per game played. Of those player-seasons, what percent of games were they available for on average? Use games from the 2014-2023 regular seasons.

```
# Filter for regular season data from 2014 onwards
player_data_reg_season = player_data[
    (player_data['season'] >= 2014) &
    (player_data['gametype'] == 2)
]

# Getting average points per season
avg_pts = player_data_reg_season.groupby(['season', 'player_name'])['points'].mean().reset_index()
avg_pts.columns = ['season', 'player_name', 'avg_pts']

# Count total games per season per player
total_games_per_season = player_data_reg_season.groupby(['season', 'player_name']).size().reset_index()
total_games_per_season.columns = ['season', 'player_name', 'games_listed']

# Sum missed games per season per player
total_games_per_season_missed = player_data_reg_season.groupby(['season', 'player_name'])['missed'].sum().reset_index()
total_games_per_season_missed.columns = ['season', 'player_name', 'missed_games']

# Getting percentage of games missed
games_played_season = total_games_per_season.merge(
    total_games_per_season_missed,
    on=['season', 'player_name'],
    how='left'
)
games_played_season['percent_played'] = (
    (games_played_season['games_listed'] - games_played_season['missed_games']) /
    games_played_season['games_listed']
) * 100

# Remove games where player missed
remove_games_missed = player_data_reg_season[player_data_reg_season['missed'] != 1]

# Calculate average points per game for players who didn't miss
avg_points_per_game = remove_games_missed.groupby(['season', 'player_name'])['points'].mean().reset_index()
avg_points_per_game.columns = ['season', 'player_name', 'avg_points']

# Join the data
player_season_avg = games_played_season.merge(
    avg_points_per_game,
    on=['season', 'player_name'],
    how='left'
)

# Getting those that missed less than 25% of games and that averaged 25 points or more.
filtered_players = player_season_avg[
    (player_season_avg['percent_played'] >= 25) &
    (player_season_avg['avg_points'] >= 25)
]

print("Players averaging at least 25 ppg play on average")
print(f"{filtered_players['percent_played'].mean().round(1)} % of the time")
```

Players averaging at least 25 ppg play on average
83.3 % of the time

Question 5: What % of playoff series are won by the team with home court advantage? Give your answer by round. Use playoffs series from the 2014-2022 seasons. Remember that the 2023 playoffs took place during the 2022 season (i.e. 2022-23 season).

```

# Filter for playoff data from 2014 to 2022
playoff_data = team_data[
    (team_data['season'] >= 2014) &
    (team_data['season'] <= 2022) &
    (team_data['gametype'] == 4)
]

finals_games = playoff_data['season'].nunique()
conf_finals_games = playoff_data['season'].nunique() * 2
round2_games = playoff_data['season'].nunique() * 4
round1_games = playoff_data['season'].nunique() * 8

# Create a unique series ID
playoff_data['series_id'] = (
    playoff_data['season'].astype(str) + '_' +
    playoff_data[['off_team', 'def_team']].min(axis=1) + '_' +
    playoff_data[['off_team', 'def_team']].max(axis=1)
)

# Calculate home court advantage based on the first game date where off_home = 1 within each series_id
home_court_advantage = (
    playoff_data[playoff_data['off_home'] == 1]
    .groupby('series_id')
    .apply(lambda x: x.loc[x['gamedate'].idxmin()])
    .reset_index(drop=True)
    .groupby('series_id')
    .first()
    .reset_index()[['series_id', 'off_team']]
    .rename(columns={'off_team': 'home_court_team'})
)

print(home_court_advantage)

playoff_data['winner'] = playoff_data.apply(
    lambda x: x['off_team'] if x['off_win'] == 1 else x['def_team'],
    axis=1
)

# Getting home court advantage
playoff_data = playoff_data.merge(
    home_court_advantage,
    on='series_id',
    how='left'
)

# Selecting certain datapoints
playoff_data_reduced = playoff_data[
    ['series_id', 'season', 'nbagameid', 'gamedate', 'winner', 'home_court_team']
].copy()

# Grouping the data and rolling it up to see when series starts and home court team roll up
playoff_data_reduced_sorted = (
    playoff_data_reduced
    .sort_values('gamedate')
    .groupby(['series_id', 'season', 'nbagameid'], as_index=False)
    .agg({
        'gamedate': 'first',
        'winner': 'first',
        'home_court_team': 'first'
    })
)

playoff_data_reduced_sorted = (
    playoff_data_reduced_sorted
    .sort_values('gamedate')
    .groupby(['series_id', 'season', 'nbagameid'], as_index=False)
    .agg({
        'gamedate': 'first',
        'winner': 'first',
        'home_court_team': 'first'
    })
    .groupby(['series_id', 'season', 'winner'], as_index=False)

```

```

.size()
.rename(columns={'size': 'team_wins_in_series'})
.merge(
    playoff_data_reduced_sorted[['series_id', 'season', 'nbgameid', 'gamedate', 'winner', 'home_court_team']],
    on=['series_id', 'season', 'winner'],
    how='left'
)
.sort_values(['series_id', 'season', 'gamedate'])
)

# Add cumulative wins by gamedate within each given season
playoff_data_reduced_sorted = (
    playoff_data_reduced_sorted
    .sort_values('gamedate')
    .groupby(['season', 'winner'], as_index=False)
    .apply(lambda x: x.assign(cumulative_wins=x.groupby('winner').cumcount() + 1))
    .reset_index(drop=True)
)

# Getting playoff round series
playoff_data_reduced_sorted['playoff_round'] = playoff_data_reduced_sorted['cumulative_wins'].apply(
    lambda x: 'round_1' if 0 <= x <= 4
    else 'round_2' if 4 < x <= 8
    else 'conf_finals' if 8 < x <= 12
    else 'finals' if 12 < x <= 16
    else 'Unknown'
)

# Splitting data to who won each round
round_1_winners = playoff_data_reduced_sorted[playoff_data_reduced_sorted['cumulative_wins'] == 4]
round_2_winners = playoff_data_reduced_sorted[playoff_data_reduced_sorted['cumulative_wins'] == 8]
conf_finals_winners = playoff_data_reduced_sorted[playoff_data_reduced_sorted['cumulative_wins'] == 12]
finals_winners = playoff_data_reduced_sorted[playoff_data_reduced_sorted['cumulative_wins'] == 16]

# Did the home team win?
home_finals_wins = pd.DataFrame({
    'home_court_wins': [(finals_winners['winner'] == finals_winners['home_court_team']).sum()],
    'finals_games': [len(finals_winners)]
})
home_finals_wins['percentage_home_court_wins'] = (
    home_finals_wins['home_court_wins'] / home_finals_wins['finals_games'] * 100
).round(1)

print(home_finals_wins)

# Conf Finals
conf_finals_wins = pd.DataFrame({
    'home_court_wins': [(conf_finals_winners['winner'] == conf_finals_winners['home_court_team']).sum()],
    'conf_finals_games': [len(conf_finals_winners)]
})

home_conf_finals_wins = conf_finals_wins.copy()
home_conf_finals_wins['percentage_home_court_wins'] = (
    home_conf_finals_wins['home_court_wins'] / home_conf_finals_wins['conf_finals_games'] * 100
).round(1)

print(home_conf_finals_wins)

# Second Round
second_round_wins = pd.DataFrame({
    'home_court_wins': [(round_2_winners['winner'] == round_2_winners['home_court_team']).sum()],
    'round_2_games': [len(round_2_winners)]
})

home_second_round_wins = second_round_wins.copy()
home_second_round_wins['percentage_home_court_wins'] = (
    home_second_round_wins['home_court_wins'] / round_2_games * 100
).round(1)

print(home_second_round_wins)

# First Round
first_round_wins = pd.DataFrame({

```

```

    'home_court_wins': [(round_1_winners['winner'] == round_1_winners['home_court_team']).sum()],
    'round_1_games': [len(round_1_winners)]
})

home_first_round_wins = first_round_wins.copy()
home_first_round_wins['percentage_home_court_wins'] = (
    home_first_round_wins['home_court_wins'] / round1_games * 100
).round(1)

print(home_first_round_wins)

```

```

      series_id home_court_team
0    2014_ATL_BKN           ATL
1    2014_ATL_CLE           ATL
2    2014_ATL_WAS           ATL
3    2014_BOS_CLE           CLE
4    2014_CHI_CLE           CLE
..      ...
130  2022_GSW_SAC           SAC
131  2022_LAC_PHX           PHX
132  2022_LAL_MEM           MEM
133  2022_MIA_MIL           MIL
134  2022_MIA_NYK           NYK

[135 rows x 2 columns]
      home_court_wins  finals_games  percentage_home_court_wins
0              7           9              77.8
      home_court_wins  conf_finals_games  percentage_home_court_wins
0              10           18              55.6
      home_court_wins  round_2_games  percentage_home_court_wins
0              23           36              63.9
      home_court_wins  round_1_games  percentage_home_court_wins
0              61           72              84.7

```

Question 6: Among teams that had at least a +5.0 net rating in the regular season, what percent of them made the second round of the playoffs the following year? Among those teams, what percent of their top 5 total minutes played players (regular season) in the +5.0 net rating season played in that 2nd round playoffs series? Use the 2014-2021 regular seasons to determine the +5 teams and the 2015-2022 seasons of playoffs data.


```

team_data_reg_season['ORTG'] = team_data_reg_season['points'] / (team_data_reg_season['possessions']/100)
team_data_reg_season['DRTG'] = team_data_reg_season['points'] / (team_data_reg_season['possessions']/100)
team_data_reg_season.head()

# Splitting data into ORTG and DRTG
average_ORTG_per_season = (
    team_data_reg_season
    .groupby(['season', 'off_team'])['ORTG']
    .mean()
    .reset_index()
    .rename(columns={'off_team': 'team'})
)
average_ORTG_per_season['season_id'] = (
    average_ORTG_per_season['season'].astype(str) + '_' +
    average_ORTG_per_season['team']
)

print(average_ORTG_per_season)

average_DRTG_per_season = (
    team_data_reg_season
    .groupby(['season', 'def_team'])['DRTG'] # Fixed: should be DRTG, not ORTG
    .mean()
    .reset_index()
    .rename(columns={'def_team': 'team'})
)
average_DRTG_per_season['season_id'] = (
    average_DRTG_per_season['season'].astype(str) + '_' +
    average_DRTG_per_season['team']
)

print(average_DRTG_per_season)

# Join ORTG and DRTG
RTG_per_season = average_DRTG_per_season.merge(
    average_ORTG_per_season,
    on=['season', 'season_id', 'team'],
    how='inner'
)[['season', 'season_id', 'DRTG', 'ORTG']]

# Calculate Net Rating
RTG_per_season['Net_Rating'] = RTG_per_season['ORTG'] - RTG_per_season['DRTG']

RTG_per_season.head()

# Getting top 5 net ratings (2014-2021)
net_5_rtg = RTG_per_season[
    (RTG_per_season['season'] >= 2014) &
    (RTG_per_season['season'] <= 2021) &
    (RTG_per_season['Net_Rating'] >= 5.0)
]

net_5_rtg

# Create previous_season_id for round_1_winners (from previous code)
round_1_winners['previous_season_id'] = (
    (round_1_winners['season'] - 1).astype(str) + '_' +
    round_1_winners['winner']
)

round_1_winners

# Join to see which +5.0 teams made round 2 the following year
prev_season_to_round_2 = net_5_rtg.merge(
    round_1_winners,
    left_on='season_id',
    right_on='previous_season_id',
    how='left'
)

```

```
prev_season_to_round_2
```

```
# Calculate percentage of teams that made round 2
```

```
percentage_unique = pd.DataFrame({  
    'percentage': [  
        round((prev_season_to_round_2['series_id'].nunique()) / len(prev_season_to_round_2) * 100, 1)  
    ]  
})
```

```
prev_season_to_round_2
```

```
print(f"Teams with a net rating above 5 made the 2nd round of the next year's playoffs {percentage_unique}% of the time")
```

	season	team	ORTG	season_id
0	2014	ATL	109.234965	2014_ATL
1	2014	BKN	104.343491	2014_BKN
2	2014	BOS	104.759001	2014_BOS
3	2014	CHA	100.766933	2014_CHA
4	2014	CHI	107.439368	2014_CHI
..	...	...	...	...
295	2023	SAC	117.156246	2023_SAC
296	2023	SAS	110.513656	2023_SAS
297	2023	TOR	112.873964	2023_TOR
298	2023	UTA	115.717770	2023_UTA
299	2023	WAS	111.122589	2023_WAS

```
[300 rows x 4 columns]
```

	season	team	DRTG	season_id
0	2014	ATL	103.027815	2014_ATL
1	2014	BKN	107.298333	2014_BKN
2	2014	BOS	104.710829	2014_BOS
3	2014	CHA	104.138981	2014_CHA
4	2014	CHI	104.043167	2014_CHI
..	...	...	...	...
295	2023	SAC	115.994579	2023_SAC
296	2023	SAS	116.785945	2023_SAS
297	2023	TOR	119.347224	2023_TOR
298	2023	UTA	120.725530	2023_UTA
299	2023	WAS	120.454284	2023_WAS

```
[300 rows x 4 columns]
```

```

prev_season_to_round_2_winners = prev_season_to_round_2[prev_season_to_round_2['winner'].notna()].copy()

prev_season_to_round_2_winners

# Getting minutes played
player_data_reg_season['minutes_played'] = player_data_reg_season['seconds'] / 60
# Group by season, team, and player_name and calculate total_minutes
player_data_reg_season_minutes = (
    player_data_reg_season
    .groupby(['season', 'team', 'player_name'])['minutes_played']
    .sum()
    .reset_index()
    .rename(columns={'minutes_played': 'total_minutes'})
)
player_data_reg_season_minutes['season_id'] = (
    player_data_reg_season_minutes['season'].astype(str) + '_' +
    player_data_reg_season_minutes['team']
)

player_data_reg_season_minutes

# Top five players minutes played per team (2014-2021)
top_five_minutes_per_team = (
    player_data_reg_season_minutes[
        (player_data_reg_season_minutes['season'] >= 2014) &
        (player_data_reg_season_minutes['season'] <= 2021)
    ]
    .groupby(['season', 'team'])
    .apply(lambda x: x.nlargest(5, 'total_minutes'))
    .reset_index(drop=True)
)

top_five_minutes_per_team

# Joining the data
top_5_minutes_net_rtg = prev_season_to_round_2_winners.merge(
    top_five_minutes_per_team,
    on='season_id',
    how='left'
)

top_5_minutes_net_rtg['playoff_id'] = (
    top_5_minutes_net_rtg['season_y'].astype(int).astype(str) + '_' +
    top_5_minutes_net_rtg['team']
)

players_net_rating = top_5_minutes_net_rtg[['season_id', 'Net_Rating', 'team', 'player_name', 'playoff_id']]

players_net_rating['Top_5'] = 'yes'

players_net_rating

player_data_playoffs_round2 = (
    player_data[
        (player_data['season'] >= 2015) &
        (player_data['season'] <= 2022) &
        (player_data['gametype'] == 4) &
        (player_data['missed'] == 0) &
        (player_data['seconds'] > 0)
    ][['season', 'team', 'player_name']]
    .groupby(['season', 'team', 'player_name'])
    .first()
    .reset_index()
    .copy()
)

player_data_playoffs_round2['playoff_id'] = (player_data_playoffs_round2['season'].astype(str) + '_' +
    player_data_playoffs_round2['team'])

player_data_playoffs_round2

```

```
# Find columns that are in both DataFrames except the join keys
common_cols = set(players_net_rating.columns).intersection(
    set(player_data_playoffs_round2.columns)
) - {'playoff_id', 'player_name'}

# Drop these columns from the right DataFrame before merging
player_data_playoffs_round2_no_dupes = player_data_playoffs_round2.drop(columns=common_cols)

# Merge using the de-duplicated right DataFrame
merged_data = players_net_rating.merge(
    player_data_playoffs_round2_no_dupes,
    left_on=['playoff_id', 'player_name'],
    right_on=['playoff_id', 'player_name'],
    how='left'
)

merged_data

total_rows = len(merged_data)
percentage = ((total_rows - merged_data['season'].isna().sum()) / total_rows) * 100

print(f"The % of players who were top 5 in minutes who played into the 2nd round of the next year's playoffs is {percentage:.1f}%")

The % of players who were top 5 in minutes who played into the 2nd round of the next year's playoffs is 79.0%
```

Model below applies training and testing all previous regular season and playoff series to then apply it to 2023 season and identify who will win the 2024 playoffs

```
#Western and Eastern Conference Teams
east_teams = [
    "ATL", "BOS", "BKN", "CHA", "CHI", "CLE", "DET", "IND",
    "MIA", "MIL", "NYK", "ORL", "PHI", "TOR", "WAS"
]

west_teams = [
    "DAL", "DEN", "GSW", "HOU", "LAC", "LAL", "MEM", "MIN",
    "NOP", "OKC", "PHX", "POR", "SAC", "SAS", "UTA"
]

#Load in the data
team_df = team_data
player_df = player_data
```

team_df																				
...	↑↓	...	↑↓	...	↑↓	n	...	↑↓	gamedate	...	↑↓	offensivenbate...	...	↑↓	off_team_name	...	↑↓	...	↑↓	...
	0		2016		2	21600495			2016-12-30T00:00:00.000			1610612740			New Orleans Pelicans			NOP		
	1		2016		2	21600495			2016-12-30T00:00:00.000			1610612752			New York Knicks			NYK		
	2		2021		2	22100943			2022-03-03T00:00:00.000			1610612742			Dallas Mavericks			DAL		
	3		2021		2	22100943			2022-03-03T00:00:00.000			1610612744			Golden State Warriors			GSW		
	4		2016		2	21601032			2017-03-18T00:00:00.000			1610612741			Chicago Bulls			CHI		
	5		2016		2	21601032			2017-03-18T00:00:00.000			1610612762			Utah Jazz			UTA		
	6		2021		2	22100942			2022-03-03T00:00:00.000			1610612761			Toronto Raptors			TOR		
	7		2022		2	22200482			2022-12-23T00:00:00.000			1610612750			Minnesota Timberwolves			MIN		
	8		2016		2	21600494			2016-12-30T00:00:00.000			1610612738			Boston Celtics			BOS		
	9		2016		2	21600494			2016-12-30T00:00:00.000			1610612748			Miami Heat			MIA		
	10		2022		2	22200495			2022-12-25T00:00:00.000			1610612738			Boston Celtics			BOS		
	11		2022		2	22200495			2022-12-25T00:00:00.000			1610612749			Milwaukee Bucks			MIL		
	12		2017		2	21700768			2018-02-02T00:00:00.000			1610612747			Los Angeles Lakers			LAL		
	13		2017		2	21700768			2018-02-02T00:00:00.000			1610612751			Brooklyn Nets			BKN		
	14		2023		2	22300197			2023-11-15T00:00:00.000			1610612750			Minnesota Timberwolves			MIN		
	15		2016		2	21600493			2016-12-30T00:00:00.000			1610612751			Brooklyn Nets			BKN		

Rows: 2,380 ⚠ Truncated from 27,144 rows 🔗 Expand

```
#Offensive stats
off_regular_stats = team_df[team_df["gametype"] == 2].copy()
off_regular_stats.rename(columns = {'off_win': 'win', 'defensivefouls': 'foulsdrawn'}, inplace=True)

#Some advanced stats
off_regular_stats["PPA"] = off_regular_stats["shotattemptpoints"] / off_regular_stats["shotattempts"]
off_regular_stats["TOV_pct"] = off_regular_stats["turnovers"] / (off_regular_stats["shotattempts"] +
off_regular_stats["turnovers"])
off_regular_stats["FG_pct"] = off_regular_stats['fgmade']/ off_regular_stats['fgattempted']
off_regular_stats['fg3_pct'] = off_regular_stats['fg3made'] / off_regular_stats['fg3attempted']
off_regular_stats['ft_pct'] = off_regular_stats['ftmade'] / off_regular_stats['ftattempted']
off_regular_stats['ts_pct'] = off_regular_stats['points'] / (2* (off_regular_stats['fgattempted'] +
0.44*off_regular_stats['ftattempted'])) * 100
off_regular_stats["homewin"] = ((off_regular_stats["off_home"] == 1) & (off_regular_stats["win"] == 1)
).astype(int)
```

```
off_regular_stats
#Groupby perseason to get relevant by season data
off_stats_per_season = off_regular_stats.groupby(["season", "off_team_name", "off_team"]).agg({
    "points": "mean",
    "possessions": "mean",
    "turnovers": "mean",
    "shotattempts": "mean",
    "shotattemptpoints": "mean",
    "win": "sum",
    "homewin": "sum",
    "fg3attempted": "mean",
    "ftattempted": "mean",
    "reboffensive": "mean",
    "assists": "mean",
    "foulsdrawn": "mean",
    "eFG": "mean",
    "PPA": "mean",
    "TOV_pct": "mean",
    "FG_pct": "mean",
    "fg3_pct": "mean",
    "ft_pct": "mean",
    "ts_pct": "mean"
}).reset_index()
```

off_stats_per_season																						
...	↑↓	...	↑↓	off_team_name	...	↑↓	...	↑↓	points	...	↑↓	possessi...	...	↑↓	turnov...	...	↑↓	shotatte...	...	↑↓	shotatte	
0		2014		Atlanta Hawks			ATL		102.5487804878			94.0243902439			14.2317073171			88.3292682927				
1		2014		Boston Celtics			BOS		101.3658536585			96.7926829268			13.8170731707			94.8902439024				
2		2014		Brooklyn Nets			BKN		98.0243902439			94.0365853659			13.8170731707			90.4146341463				
3		2014		Charlotte Hornets			CHA		94.1585365854			93.5487804878			11.9024390244			91.987804878				
4		2014		Chicago Bulls			CHI		100.7926829268			93.9268292683			13.9756097561			91.487804878				
5		2014		Cleveland Cavaliers			CLE		103.1341463415			92.9512195122			14.2804878049			90				
6		2014		Dallas Mavericks			DAL		105.2195121951			96.3780487805			12.9512195122			93.0487804878				
7		2014		Denver Nuggets			DEN		101.4634146341			97.256097561			14.2195121951			95.756097561				
8		2014		Detroit Pistons			DET		98.5			93.512195122			13.4024390244			93.487804878				
9		2014		Golden State Warriors			GSW		109.9512195122			98.5487804878			14.4512195122			94.3048780488				
10		2014		Houston Rockets			HOU		103.9268292683			97.2317073171			16.6585365854			92.0731707317				
11		2014		Indiana Pacers			IND		97.243902439			93.7804878049			13.987804878			89.9024390244				
12		2014		LA Clippers			LAC		106.7195121951			94.9024390244			12.3414634146			90.3536585366				
13		2014		Los Angeles Lakers			LAL		98.4268292683			95.2195121951			13.243902439			93.9390243902				
14		2014		Memphis Grizzlies			MEM		98.3170731707			92.9634146341			13.3536585366			90.0487804878				
15		2014		Miami Heat			MIA		94.6829268293			90.9756097561			14.7926829268			85.243902439				

Rows: 300

Expand

```

#Get defensive data now and adjust column names to defensive type of names
def_regular_stats = off_regular_stats
def_regular_stats.rename(columns = {
    "fg3attempted": "fg3_concede",
    "ftattempted": "ft_concede",
    "reboffensive": "rebs_concede",
    "stealsagainst": "steals",
    "turnovers": "to_forced",
    "blockagainst": "blocks",
    "foulsdrawn": "fouls",
    "eFG": "deFG",
    "FG_pct": "fg_pct_concede", # This will be renamed to lowercase 'fg_pct_concede'
    "fg3_pct": "fg3_pct_concede",
    "ts_pct": "ts_pct_concede",
    "points": "points_concede"
}, inplace = True)

# Check the columns after renaming to ensure correct names
print(def_regular_stats.columns)

#Groupby perseason to get relevant by season data
def_stats_per_season = def_regular_stats.groupby(["season", "def_team_name", "def_team"]).agg({
    "points_concede": "mean",
    "to_forced": "mean",
    "fg3_concede": "mean",
    "ft_concede": "mean",
    "rebs_concede": "mean",
    "steals": "mean",
    "blocksagainst": "mean",
    "fouls": "mean",
    "deFG": "mean",
    "fg_pct_concede": "mean", # Changed from 'FG_pct_concede' to 'fg_pct_concede'
    "fg3_pct_concede": "mean",
    "ts_pct_concede": "mean"
}).reset_index()

def_stats_per_season

```

```

Index(['season', 'gametype', 'nbagameid', 'gamedate', 'offensivenbteamid',
      'off_team_name', 'off_team', 'off_home', 'win', 'defensivenbteamid',
      'def_team_name', 'def_team', 'def_home', 'def_win', 'fg2made',
      'fg2missed', 'fg2attempted', 'fg3made', 'fg3missed', 'fg3_concede',
      'fgmade', 'fgmissed', 'fgattempted', 'ftmade', 'ftmissed', 'ft_concede',
      'rebs_concede', 'rebdefensive', 'reboundchance', 'assists', 'steals',
      'to_forced', 'blocksagainst', 'fouls', 'offensivefouls',
      'shootingfoulsdrawn', 'possessions', 'points_concede', 'shotattempts',
      'andones', 'shotattemptpoints', 'deFG', 'PPA', 'TOV_pct',
      'fg_pct_concede', 'fg3_pct_concede', 'ft_pct', 'ts_pct_concede',
      'homewin'],
      dtype='object')

```

...	↑↓	...	↑↓	def_team_name	...	↑↓	...	↑↓	points_c...	...	↑↓	to_for...	...	↑↓	fg3_co...	...	↑↓	ft_con...	...	↑↓	rebs_c...	...
0		2014		Atlanta Hawks			ATL		97.1219512195			16.0609756098			25.756097561			20.1219512195			14.8414	
1		2014		Boston Celtics			BOS		101.2073170732			15.1097560976			22			23.2804878049			14.3170	
2		2014		Brooklyn Nets			BKN		100.9024390244			13.8658536585			23.7073170732			21.3170731707			14.5853	
3		2014		Charlotte Hornets			CHA		97.3292682927			12.5975609756			22.1463414634			20.4024390244			12.4512	
4		2014		Chicago Bulls			CHI		97.7926829268			12.1829268293			19.5731707317			21.012195122			15.0243	
5		2014		Cleveland Cavaliers			CLE		98.6585365854			13.3170731707			22.5365853659			19.8658536585			13.9024	
6		2014		Dallas Mavericks			DAL		102.3170731707			16.3292682927			24.8048780488			22.9146341463			15.7195	
7		2014		Denver Nuggets			DEN		105.012195122			14.3536585366			22.2317073171			26.9268292683			14.6951	
8		2014		Detroit Pistons			DET		99.4756097561			13.9756097561			21.8170731707			20.6951219512			13.7195	
9		2014		Golden State Warriors			GSW		99.8536585366			16.2073170732			21.4268292683			24.512195122				
10		2014		Houston Rockets			HOU		100.487804878			16.3658536585			22.9390243902			23.5609756098			15.7804	
11		2014		Indiana Pacers			IND		96.9390243902			12.756097561			20.8658536585			23.7682926829			13.1219	
12		2014		LA Clippers			LAC		100.1341463415			14.2804878049			23.8902439024			25.6707317073			13.9634	
13		2014		Los Angeles Lakers			LAL		105.2804878049			13.0243902439			23.4268292683			25.8048780488			13.3048	
14		2014		Memphis Grizzlies			MEM		95.0731707317			15.3170731707			23.0365853659			20.1341463415			13.6829	

15	2014	Miami Heat	MIA	97.2804878049	14.8292682927	23.012195122	23.2317073171	14.6097
----	------	------------	-----	---------------	---------------	--------------	---------------	---------

Rows: 300 Expand

```
# Join both stats for a holistic stats by year
team_stats_season = off_stats_per_season.merge(
    def_stats_per_season,
    left_on=["season", "off_team"],
    right_on=["season", "def_team"],
    how="inner", # or "left" if you want to keep all offensive rows
)

# Get away win total per season
team_stats_season['awaywin'] = team_stats_season['win'] - team_stats_season['homewin']
team_stats_season['win_pct'] = team_stats_season['win']/82
team_stats_season['season_id'] = (team_stats_season['season']).astype(str) + '_' + team_stats_season['off_team']

#Now join with our net rating data to get comprehensive data that is solid before we bring in certain player data elements
team_stats_season

season_net = team_stats_season.merge(
    RTG_per_season,
    on = "season_id",
    how = "inner"
)
season_net = season_net.drop(columns=["season_y", "def_team_name", "def_team"])
season_net
```

...	↑↓	...	↑↓	off_team_name	...	↑↓	...	↑↓	points	...	↑↓	possessi...	...	↑↓	turnov...	...	↑↓	shotatte...	...	↑↓	shotat
0		2014		Atlanta Hawks			ATL		102.5487804878			94.0243902439			14.2317073171			88.3292682927			
1		2014		Boston Celtics			BOS		101.3658536585			96.7926829268			13.8170731707			94.8902439024			
2		2014		Brooklyn Nets			BKN		98.0243902439			94.0365853659			13.8170731707			90.4146341463			
3		2014		Charlotte Hornets			CHA		94.1585365854			93.5487804878			11.9024390244			91.987804878			
4		2014		Chicago Bulls			CHI		100.7926829268			93.9268292683			13.9756097561			91.487804878			
5		2014		Cleveland Cavaliers			CLE		103.1341463415			92.9512195122			14.2804878049			90			
6		2014		Dallas Mavericks			DAL		105.2195121951			96.3780487805			12.9512195122			93.0487804878			
7		2014		Denver Nuggets			DEN		101.4634146341			97.256097561			14.2195121951			95.756097561			
8		2014		Detroit Pistons			DET		98.5			93.512195122			13.4024390244			93.487804878			
9		2014		Golden State Warriors			GSW		109.9512195122			98.5487804878			14.4512195122			94.3048780488			
10		2014		Houston Rockets			HOU		103.9268292683			97.2317073171			16.6585365854			92.0731707317			
11		2014		Indiana Pacers			IND		97.243902439			93.7804878049			13.987804878			89.9024390244			
12		2014		LA Clippers			LAC		106.7195121951			94.9024390244			12.3414634146			90.3536585366			
13		2014		Los Angeles Lakers			LAL		98.4268292683			95.2195121951			13.243902439			93.9390243902			
14		2014		Memphis Grizzlies			MEM		98.3170731707			92.9634146341			13.3536585366			90.0487804878			
15		2014		Miami Heat			MIA		94.6829268293			90.9756097561			14.7926829268			85.243902439			

Rows: 300 Expand

```

#Bring in player data to eventually align to season_net data
player_df["minutes"] = player_df["seconds"] / 60
player_df['player_fg3_pct'] = player_df['fg3made'] / player_df['fg3attempted']
player_df['player_fg_pct'] = player_df['fgmade'] / player_df['fgattempted']
player_df["player_PPA"] = player_df["shotattemptpoints"] / player_df["shotattempts"]
player_df['player_ft_pct'] = player_df['ftmade'] / player_df['ftattempted']
player_df['player_oreb_pct'] = player_df['reboffensive'] / player_df['offensivereboundchances']
player_df['player_dreb_pct'] = player_df['rebdefensive'] / player_df['defensivereboundchances']
player_df['player_usg'] = (player_df['fgattempted'] + player_df['turnovers']) / (player_df['teamshotattempts'] + player_df['teamturnovers'])
player_df['plus_minus'] = player_df['teampoints'] - player_df['opponentteampoints']
player_df['player_ts_pct'] = player_df['points'] / (2 * (player_df['fgattempted'] + 0.44 * player_df['ftattempted'])) * 100
player_df['player_efg'] = (player_df['fgmade'] + 0.5 * player_df['fg3made']) / player_df['fgattempted']

#Split into postseason v regular season for comparison analysis possibly later
player_regular = player_df[player_df["gametype"] == 2].copy()
player_playoffs = player_df[player_df["gametype"] == 4].copy()

player_season_stats = player_regular.groupby(
    ["season", "nbateamid", "nbapersonid", "player_name", "team_name"]
).agg({
    "minutes": "mean",
    "points": "mean",
    "starter": "sum",
    "missed": "sum",
    "fgattempted": "mean",
    "fg3attempted": "mean",
    "ftattempted": "mean",
    "assists": "mean",
    "steals": "mean",
    "blocks": "mean",
    "player_fg3_pct": "mean",
    "player_fg_pct": "mean",
    "player_ft_pct": "mean",
    "player_oreb_pct": "mean",
    "player_dreb_pct": "mean",
    "plus_minus": "mean",
    "player_ts_pct": "mean",
    "player_efg": "mean",
    "player_usg": "mean"
}).reset_index()

# Rank players by minutes (rotation)
player_season_stats = player_season_stats.sort_values(
    ["season", "nbateamid", "minutes"], ascending=False)
#Get ranking
player_season_stats["rank"] = player_season_stats.groupby(
    ["season", "nbateamid"]
)[["minutes"]].rank(method="first", ascending=False)

# Top 8 rotation players
rotation = player_season_stats[player_season_stats["rank"] <= 8]

#Another groupby in order to aggregate just the top 8
player_reg_kpi = rotation.groupby(['season', 'nbateamid', 'team_name']).agg({
    "points": ["sum", "max"],
    "minutes": ["mean", "max"],
    "starter": "max",
    "missed": "mean",
    "assists": ["sum", "max"],
    "steals": ["sum", "max"],
    "blocks": ["sum", "max"],
    "player_fg3_pct": ["mean", "max"],
    "player_fg_pct": ["mean", "max"],
    "player_oreb_pct": ["mean", "max"],
    "player_dreb_pct": ["mean", "max"],
    "plus_minus": ["sum", "max"],
    "player_ts_pct": ["mean", "max"],
    "player_efg": ["mean", "max"],
    "player_usg": ["mean", "max"]
})

```



```
player_reg_kpi.columns = ["_".join(col) for col in player_reg_kpi.columns]
```

```
player_reg_kpi = player_reg_kpi.reset_index()
```

player_reg_kpi

...	↑↓	...	↑↓	n...	...	↑↓	team_name	...	↑↓	points_s...	...	↑↓	points...	...	↑↓	minute...	...	↑↓	minute...	...	↑↓	sta...	...
123		2018		1610612740			New Orleans Pelicans			98.8920177384			19.0853658537			21.391618887			29.2395934959				
290		2023		1610612757			Portland Trail Blazers			82.4611280488			15.65625			21.5115796494			27.3522916667				
270		2023		1610612737			Atlanta Hawks			103.9268292683			21.3780487805			24.4246239837			33.8895528455				
272		2023		1610612739			Cleveland Cavaliers			94.7195121951			17.8414634146			23.9474059959			29.7170121951				
281		2023		1610612748			Miami Heat			91.9514522158			16.6707317073			23.8468581487			29.4117479675				
269		2022		1610612766			Charlotte Hornets			87.319619405			16.2073170732			22.8130042563			28.965203252				
212		2021		1610612739			Cleveland Cavaliers			89.4794166989			17.9634146341			23.1741707129			29.5817479675				
267		2022		1610612764			Washington Wizards			92.380699894			18.3536585366			22.4464686064			27.260203252				
221		2021		1610612748			Miami Heat			87.9512195122			16.6707317073			23.2014456301			26.227601626				
214		2021		1610612741			Chicago Bulls			96.0790031813			25.8292682927			23.3610051034			33.4026829268				
71		2016		1610612748			Miami Heat			87.5853658537			18.0853658537			23.9322433943			30.6530081301				
210		2021		1610612737			Atlanta Hawks			97.6256097561			26.2804878049			23.2507309451			32.2826422764				
252		2022		1610612749			Milwaukee Bucks			93.4024390244			23.8902439024			22.3220426829			28.8715853659				
256		2022		1610612753			Orlando Magic			85.9328198545			18.1097560976			21.5617541185			31.7497764228				
174		2019		1610612761			Toronto Raptors			93.4861111111			19.0416666667			23.4681481481			29.2627777778				
273		2023		1610612740			New Orleans Pelicans			94.5853658537			19.5243902439			22.8621646341			28.2587398374				

Rows: 300

Expand

```
#join data to team level for a long dataframe by one line
```

```
season_reg_player = season_net.merge(  
    player_reg_kpi,  
    left_on= ["season_x", "off_team_name"],  
    right_on= ["season", "team_name"],  
    how = "inner")
```

```
season_reg_player.drop(columns = ['season', 'team_name'])  
season_reg_player
```

...	↑↓	...	↑↓	off_team_name	...	↑↓	...	↑↓	points	...	↑↓	possessi...	...	↑↓	turnov...	...	↑↓	shotatte...	...	↑↓	shotat	...
0		2014		Atlanta Hawks			ATL		102.5487804878			94.0243902439			14.2317073171			88.3292682927				
1		2014		Boston Celtics			BOS		101.3658536585			96.7926829268			13.8170731707			94.8902439024				
2		2014		Brooklyn Nets			BKN		98.0243902439			94.0365853659			13.8170731707			90.4146341463				
3		2014		Charlotte Hornets			CHA		94.1585365854			93.5487804878			11.9024390244			91.987804878				
4		2014		Chicago Bulls			CHI		100.7926829268			93.9268292683			13.9756097561			91.487804878				
5		2014		Cleveland Cavaliers			CLE		103.1341463415			92.9512195122			14.2804878049			90				
6		2014		Dallas Mavericks			DAL		105.2195121951			96.3780487805			12.9512195122			93.0487804878				
7		2014		Denver Nuggets			DEN		101.4634146341			97.256097561			14.2195121951			95.756097561				
8		2014		Detroit Pistons			DET		98.5			93.512195122			13.4024390244			93.487804878				
9		2014		Golden State Warriors			GSW		109.9512195122			98.5487804878			14.4512195122			94.3048780488				
10		2014		Houston Rockets			HOU		103.9268292683			97.2317073171			16.6585365854			92.0731707317				
11		2014		Indiana Pacers			IND		97.243902439			93.7804878049			13.987804878			89.9024390244				
12		2014		LA Clippers			LAC		106.7195121951			94.9024390244			12.3414634146			90.3536585366				
13		2014		Los Angeles Lakers			LAL		98.4268292683			95.2195121951			13.243902439			93.9390243902				
14		2014		Memphis Grizzlies			MEM		98.3170731707			92.9634146341			13.3536585366			90.0487804878				
15		2014		Miami Heat			MIA		94.6829268293			90.9756097561			14.7926829268			85.243902439				

Rows: 300

Expand

```
#Identify the data and split into regular season and playoffs
playoff_df = team_df[team_df["gametype"]==4].copy()
playoff_df = playoff_df.sort_values(["season", "nbagameid", "off_team_name"])

# Create matchup key (order independent)
playoff_df["matchup"] = playoff_df.apply(
    lambda x: "_".join(sorted([x["off_team_name"], x["def_team_name"]])),
    axis=1
)

# Win flag
playoff_df.rename(columns = {"off_win": "win"}, inplace = True)
```

playoff_df

	...	...	n	gamedate	offensivenbate...	off_team_name		
24927	2004	4	40400101	2005-04-24T00:00:00.000	1610612751	Brooklyn Nets	BKN	
24926	2004	4	40400101	2005-04-24T00:00:00.000	1610612748	Miami Heat	MIA	
24930	2004	4	40400102	2005-04-26T00:00:00.000	1610612751	Brooklyn Nets	BKN	
24929	2004	4	40400102	2005-04-26T00:00:00.000	1610612748	Miami Heat	MIA	
24923	2004	4	40400103	2005-04-28T00:00:00.000	1610612751	Brooklyn Nets	BKN	
24922	2004	4	40400103	2005-04-28T00:00:00.000	1610612748	Miami Heat	MIA	
24920	2004	4	40400104	2005-05-01T00:00:00.000	1610612751	Brooklyn Nets	BKN	
24919	2004	4	40400104	2005-05-01T00:00:00.000	1610612748	Miami Heat	MIA	
24918	2004	4	40400111	2005-04-23T00:00:00.000	1610612765	Detroit Pistons	DET	
24917	2004	4	40400111	2005-04-23T00:00:00.000	1610612755	Philadelphia 76ers	PHI	
24916	2004	4	40400112	2005-04-26T00:00:00.000	1610612765	Detroit Pistons	DET	
24915	2004	4	40400112	2005-04-26T00:00:00.000	1610612755	Philadelphia 76ers	PHI	
24914	2004	4	40400113	2005-04-29T00:00:00.000	1610612765	Detroit Pistons	DET	
24913	2004	4	40400113	2005-04-29T00:00:00.000	1610612755	Philadelphia 76ers	PHI	
24910	2004	4	40400114	2005-05-01T00:00:00.000	1610612765	Detroit Pistons	DET	
24909	2004	4	40400114	2005-05-01T00:00:00.000	1610612755	Philadelphia 76ers	PHI	

Rows: 2,325 Truncated from 3,186 rows Expand

```
#Get one line per team, home team is always team_name_a and then prepare to join data into here
reduced_playoff = playoff_df[['season', 'gametype', 'nbagameid', 'gamedate', 'off_team_name', 'off_team', 'off_home', 'win',
                              'def_team_name', 'def_team', 'matchup']]
reduced_playoff = reduced_playoff[reduced_playoff['off_home'] == 1]
reduced_playoff.rename(columns = {'off_team_name': 'team_name_a', 'off_team': 'team_a', 'off_home': 'home_game', 'def_team_name':
                              'team_name_b',
                              'def_team': 'team_b', 'win': 'target'}, inplace = True)
reduced_playoff.head(n=1)
```

...	↑↓	...	↑↓	...	↑↓	n	...	↑↓	gamedate	...	↑↓	tea...	...	↑↓	...	↑↓	h	...	↑↓	...	↑↓	team_...	...	↑↓	...
24926		2004		4		40400101			2005-04-24T00:00:00.000			Miami Heat			MIA		1		1		Brooklyn Nets			BKN	

Rows: 1 Expand

```
# Split data into pre-2023 regular season and 2023 data
train_reg = season_reg_player[season_reg_player['season'] < 2023].copy()
model_reg = season_reg_player[season_reg_player['season'] == 2023].copy()
#Get playoff data to merge with regular season stats
playoff_reg_merge = reduced_playoff.merge(
    train_reg, left_on=['season', 'team_name_a'], right_on=['season', 'team_name']
).merge(
    train_reg, left_on=['season', 'team_name_b'], right_on=['season', 'team_name'],
    suffixes=("_home", "_away")
)

playoff_reg_merge.head(n=1)
```

...	↑↓	...	↑↓	...	↑↓	n	...	↑↓	gamedate	...	↑↓	team_...	...	↑↓	...	↑↓	h.	...	↑↓	...	↑↓	team_...	...	↑↓	
0		2014		4		41400101			2015-04-19T00:00:00.000			Atlanta Hawks			ATL		1		1		1		Brooklyn Nets		B

Rows: 1

Expand

```
#Drop all the extra columns and revisit for better transparency
playoff_reg_merge = playoff_reg_merge.drop(columns=[
    "season_x_home",
    "off_team_name_home",
    "off_team_home",
    "nbateamid_home",
    "team_name_home",
    "season_x_away",
    "off_team_name_away",
    "off_team_away",
])
#Because of issues earlier, relabel _home and _away to _a and _b
def relabel(c):
    if c.endswith("_home"):
        return c[:-5] + "_a"
    if c.endswith("_away"):
        return c[:-5] + "_b"
    return c

playoff_reg_merge = playoff_reg_merge.rename(columns=relabel)
playoff_reg_merge
concise_playoff = playoff_reg_merge.to_csv("playoff_reg_merge.csv", index=False)
```

```

import functools
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
from xgboost import XGBClassifier

ID_COLS = {
    "target", "season", "gametype", "nbagameid", "gamedate",
    "team_name_a", "team_a", "home_game", "team_name_b", "team_b", "matchup",
    "season_id_a", "season_id_b", "nbateamid_b", "team_name_b.1"
}

TEAM_NAME_ALIASES = {
    "Los Angeles Clippers": "LA Clippers",
    "Dallas Mavericks": "Dallas Mavericks", # handles typo if present
}

WEST_SEEDS = [
    "Oklahoma City Thunder", "Denver Nuggets", "Minnesota Timberwolves", "LA Clippers",
    "Dallas Mavericks", "Phoenix Suns", "Los Angeles Lakers", "New Orleans Pelicans"
]

EAST_SEEDS = [
    "Boston Celtics", "New York Knicks", "Milwaukee Bucks", "Cleveland Cavaliers",
    "Orlando Magic", "Indiana Pacers", "Philadelphia 76ers", "Miami Heat"
]

def get_model_features(df):
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    return [c for c in numeric_cols if c not in ID_COLS]

```

```

def train_xgb_with_top_features(playoff_reg_merge, top_n=12, random_state=42):
    feature_cols = get_model_features(playoff_reg_merge)
    X = playoff_reg_merge[feature_cols]
    y = playoff_reg_merge["target"].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=random_state, stratify=y
    )

    base_model = XGBClassifier(
        n_estimators=400,
        max_depth=4,
        learning_rate=0.03,
        subsample=0.9,
        colsample_bytree=0.9,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=random_state,
    )
    base_model.fit(X_train, y_train)

    importances = pd.Series(base_model.feature_importances_, index=feature_cols).sort_values(ascending=False)
    top_features = importances.head(top_n).index.tolist()

    final_model = XGBClassifier(
        n_estimators=400,
        max_depth=4,
        learning_rate=0.03,
        subsample=0.9,
        colsample_bytree=0.9,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=random_state,
    )
    final_model.fit(X_train[top_features], y_train)

    pred_proba = final_model.predict_proba(X_test[top_features])[:, 1]
    pred_label = (pred_proba >= 0.5).astype(int)

    metrics = {
        "test_accuracy": accuracy_score(y_test, pred_label),
        "test_roc_auc": roc_auc_score(y_test, pred_proba),
    }
    return final_model, top_features, importances, metrics

```

```

def map_top_features_to_2023(top_features, matchup_df):
    mapped_cols, missing = [], []
    for f in top_features:
        candidate = f.replace("_a", "").replace("_b", "")
        source_col = f if f in matchup_df.columns else candidate
        if source_col not in matchup_df.columns:
            missing.append(f)
        else:
            mapped_cols.append((f, source_col))

    if missing:
        raise ValueError(f"Missing required features in 2023 matchup data: {missing}")

    return pd.DataFrame({train_col: matchup_df[src_col] for train_col, src_col in mapped_cols})

```

```

def build_all_matchups(model_reg):
    left = model_reg.add_suffix("_a").copy()
    right = model_reg.add_suffix("_b").copy()
    left["key"] = 1
    right["key"] = 1
    matchups = left.merge(right, on="key").drop(columns=["key"])
    matchups = matchups[matchups["off_team_a"] != matchups["off_team_b"]].copy()
    matchups["home_team"] = matchups["off_team_name_a"]
    matchups["away_team"] = matchups["off_team_name_b"]
    return matchups

def build_team_lookup(model_reg):
    df = model_reg.copy()
    df["normalized_team_name"] = df["off_team_name"].replace(TeamNameAliases)
    return df.set_index("normalized_team_name")

def build_single_matchup_row(home_team, away_team, team_lookup):
    home = team_lookup.loc[home_team].drop(labels=["off_team_name", "off_team"])
    away = team_lookup.loc[away_team].drop(labels=["off_team_name", "off_team"])
    row = {f"{col}_a": home[col] for col in home.index}
    row.update({f"{col}_b": away[col] for col in away.index})
    return pd.DataFrame([row])

def get_home_win_probability(model, top_features, team_lookup, home_team, away_team):
    row = build_single_matchup_row(home_team, away_team, team_lookup)
    X_row = map_top_features_to_2023(top_features, row)
    return float(model.predict_proba(X_row)[0, 1][0])

def get_series_schedule(team1_home_court):
    # 2-2-1-1-1
    return ["team1", "team1", "team2", "team2", "team1", "team2", "team1"] if team1_home_court else \
        ["team2", "team2", "team1", "team1", "team2", "team1", "team2"]

def pick_series_home_team(team1, team2, team_lookup):
    return float(team_lookup.loc[team1, "win"]) >= float(team_lookup.loc[team2, "win"])

```

```

def series_probabilities(team1, team2, team1_home_court, model, top_features, team_lookup):
    schedule = get_series_schedule(team1_home_court)
    game_win_probs_team1 = []
    for host in schedule:
        if host == "team1":
            p_home = get_home_win_probability(model, top_features, team_lookup, team1, team2)
            game_win_probs_team1.append(p_home)
        else:
            p_home = get_home_win_probability(model, top_features, team_lookup, team2, team1)
            game_win_probs_team1.append(1.0 - p_home)

@functools.lru_cache(maxsize=None)
def dp(t1_wins, t2_wins, game_idx):
    if t1_wins == 4:
        return {(team1, game_idx): 1.0}
    if t2_wins == 4:
        return {(team2, game_idx): 1.0}
    p_t1 = game_win_probs_team1[game_idx]
    out = {}
    for k, v in dp(t1_wins + 1, t2_wins, game_idx + 1).items():
        out[k] = out.get(k, 0.0) + p_t1 * v
    for k, v in dp(t1_wins, t2_wins + 1, game_idx + 1).items():
        out[k] = out.get(k, 0.0) + (1.0 - p_t1) * v
    return out

terminal = dp(0, 0, 0)
team1_series = sum(p for (winner, _), p in terminal.items() if winner == team1)
team2_series = 1.0 - team1_series

detail = pd.DataFrame(
    [{"winner": w, "games": g, "probability": p} for (w, g), p in terminal.items()])
).sort_values("probability", ascending=False).reset_index(drop=True)

most_likely = detail.iloc[0]
return {
    "team1_series_win_prob": float(team1_series),
    "team2_series_win_prob": float(team2_series),
    "most_likely_winner": most_likely["winner"],
    "most_likely_games": int(most_likely["games"]),
    "most_likely_outcome_probability": float(most_likely["probability"]),
    "detail": detail,
}

```

```

def run_round(round_name, pairings, model, top_features, team_lookup):
    winners, summary_rows, detail_rows = [], [], []
    for team1, team2 in pairings:
        t1_home = pick_series_home_team(team1, team2, team_lookup)
        r = series_probabilities(team1, team2, t1_home, model, top_features, team_lookup)

        winner = team1 if r["team1_series_win_prob"] >= r["team2_series_win_prob"] else team2
        winners.append(winner)

        summary_rows.append({
            "round": round_name, "team1": team1, "team2": team2,
            "team1_home_court": int(t1_home),
            "team1_series_win_prob": r["team1_series_win_prob"],
            "team2_series_win_prob": r["team2_series_win_prob"],
            "predicted_winner": winner,
            "most_likely_winner": r["most_likely_winner"],
            "most_likely_games": r["most_likely_games"],
            "most_likely_outcome_probability": r["most_likely_outcome_probability"],
        })

        d = r["detail"].copy()
        d["round"], d["team1"], d["team2"] = round_name, team1, team2
        detail_rows.append(d)

    return winners, pd.DataFrame(summary_rows), pd.concat(detail_rows, ignore_index=True)

```

```

def run_full_bracket(model, top_features, model_reg):
    team_lookup = build_team_lookup(model_reg)

    r1_west = [(WEST_SEEDS[0], WEST_SEEDS[7]), (WEST_SEEDS[1], WEST_SEEDS[6]),
               (WEST_SEEDS[2], WEST_SEEDS[5]), (WEST_SEEDS[3], WEST_SEEDS[4])]
    r1_east = [(EAST_SEEDS[0], EAST_SEEDS[7]), (EAST_SEEDS[1], EAST_SEEDS[6]),
               (EAST_SEEDS[2], EAST_SEEDS[5]), (EAST_SEEDS[3], EAST_SEEDS[4])]

    w_winners, w_sum1, w_det1 = run_round("West Round 1", r1_west, model, top_features, team_lookup)
    e_winners, e_sum1, e_det1 = run_round("East Round 1", r1_east, model, top_features, team_lookup)

    w_sf_pairings = [(w_winners[0], w_winners[3]), (w_winners[1], w_winners[2])]
    e_sf_pairings = [(e_winners[0], e_winners[3]), (e_winners[1], e_winners[2])]
    w_sf_winners, w_sum2, w_det2 = run_round("West Semifinals", w_sf_pairings, model, top_features, team_lookup)
    e_sf_winners, e_sum2, e_det2 = run_round("East Semifinals", e_sf_pairings, model, top_features, team_lookup)

    w_cf_winners, w_sum3, w_det3 = run_round("West Finals", [(w_sf_winners[0], w_sf_winners[1])], model, top_features, team_lookup)
    e_cf_winners, e_sum3, e_det3 = run_round("East Finals", [(e_sf_winners[0], e_sf_winners[1])], model, top_features, team_lookup)

    finals_winners, f_sum, f_det = run_round("NBA Finals", [(w_cf_winners[0], e_cf_winners[0])], model, top_features, team_lookup)

    bracket_summary = pd.concat([w_sum1, e_sum1, w_sum2, e_sum2, w_sum3, e_sum3, f_sum], ignore_index=True)
    bracket_detail = pd.concat([w_det1, e_det1, w_det2, e_det2, w_det3, e_det3, f_det], ignore_index=True)
    champion = finals_winners[0]
    return champion, bracket_summary, bracket_detail

model, top_features, importances, metrics = train_xgb_with_top_features(
    playoff_reg_merge=playoff_reg_merge,
    top_n=12
)

```



```

# all home/away game-level predictions across 2023 teams
all_matchups_2023 = build_all_matchups(model_reg)
X_2023 = map_top_features_to_2023(top_features, all_matchups_2023)
all_matchups_2023["home_win_probability"] = model.predict_proba(X_2023)[: , 1]
all_matchups_2023["predicted_home_win"] = (all_matchups_2023["home_win_probability"] >= 0.5).astype(int)

matchup_predictions = all_matchups_2023[
    ["home_team", "away_team", "home_win_probability", "predicted_home_win"]
].sort_values("home_win_probability", ascending=False).reset_index(drop=True)

# bracket simulation
champion, bracket_summary, bracket_detail = run_full_bracket(model, top_features, model_reg)

# =====
# PRINT METRICS / ROUND OUTPUTS
# =====

print("\n=== HOLDOUT METRICS ===")
print(f"Accuracy: {metrics['test_accuracy']:.3f}")
print(f"ROC AUC : {metrics['test_roc_auc']:.3f}")

print("\n=== TOP FEATURES ===")
for i, f in enumerate(top_features, 1):
    print(f"{i:02d}. {f}")

print("\n=== PLAYOFF MATCHUPS & SERIES PROBABILITIES ===")

def print_round(round_name, df):
    r = df[df["round"] == round_name].copy()
    if r.empty:
        print(f"\n[{round_name}] No matchups found.")
        return

    print(f"\n[{round_name}]")
    for _, row in r.iterrows():
        team1 = row["team1"]
        team2 = row["team2"]
        p1 = row["team1_series_win_prob"] * 100
        p2 = row["team2_series_win_prob"] * 100
        winner = row["predicted_winner"]
        ml_winner = row["most_likely_winner"]
        ml_games = int(row["most_likely_games"])
        ml_prob = row["most_likely_outcome_probability"] * 100

        print(f"{team1} vs {team2}")
        print(f"Series win probs: {team1} {p1:.1f}% | {team2} {p2:.1f}%")
        print(f"Predicted winner: {winner}")
        print(f"Most likely exact outcome: {ml_winner} in {ml_games} ({ml_prob:.1f}%)")
        print("")

# First round
print_round("West Round 1", bracket_summary)
print_round("East Round 1", bracket_summary)

# Second round
print_round("West Semifinals", bracket_summary)
print_round("East Semifinals", bracket_summary)

# Third round (conference finals)
print_round("West Finals", bracket_summary)
print_round("East Finals", bracket_summary)

# Finals (optional but useful)
print_round("NBA Finals", bracket_summary)

print("\n=== CHAMPION ===")
print(champion)
# Available outputs in memory:
# - matchup_predictions (all 2023 home/away matchups)
# - bracket_summary (series-level round outputs)
# - bracket_detail (all exact winner+games probabilities per series)

```

=== HOLDOUT METRICS ===

Accuracy: 0.607

ROC AUC : 0.613

=== TOP FEATURES ===

01. plus_minus_sum_b

02. win_pct_a

03. win_a

04. plus_minus_max_b

05. player_oreb_pct_max_b

06. Net_Rating_b

07. TOV_pct_a

08. ORTG_b

09. ts_pct_a

10. shotattemptpoints_b

11. ORTG_a

12. PPA_a

=== PLAYOFF MATCHUPS & SERIES PROBABILITIES ===

[West Round 1]

Oklahoma City Thunder vs New Orleans Pelicans

Series win probs: Oklahoma City Thunder 82.5% | New Orleans Pelicans 17.5%

Predicted winner: Oklahoma City Thunder

Most likely exact outcome: Oklahoma City Thunder in 5 (31.2%)

Denver Nuggets vs Los Angeles Lakers

A brief written overview of how your model works, targeted towards a decision maker in the front office without a strong statistical background.

- What you view as the strengths and weaknesses of your model.
- How you'd address the weaknesses if you had more time and/or more data.
- Apply your model to the 2024 NBA playoffs (2023 season) and create a high quality visual (a table, a plot, or a plotly) showing the 16 teams' (that made the first round) chances of advancing to each round.

```

# -----
# POST-MODEL VISUAL: 16 teams' odds to reach each round
# -----

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1) Build 16-team list from Round 1 matchups
r1 = bracket_summary[bracket_summary["round"].isin(["West Round 1", "East Round 1"])]
teams16 = sorted(set(r1["team1"]).union(set(r1["team2"])))

# 2) Initialize advancement table
adv = pd.DataFrame(index=teams16, data={
    "Reach Round 2": 0.0,      # Win Round 1
    "Reach Round 3": 0.0,      # Win Semis
    "Reach Finals": 0.0,       # Win Conference Finals
    "Win Championship": 0.0,    # Win NBA Finals
})

def assign_round_probs(round_name, col_name):
    rr = bracket_summary[bracket_summary["round"] == round_name]
    for _, row in rr.iterrows():
        adv.loc[row["team1"], col_name] = float(row["team1_series_win_prob"])
        adv.loc[row["team2"], col_name] = float(row["team2_series_win_prob"])

# Round mapping from your saved outputs
assign_round_probs("West Round 1", "Reach Round 2")
assign_round_probs("East Round 1", "Reach Round 2")
assign_round_probs("West Semifinals", "Reach Round 3")
assign_round_probs("East Semifinals", "Reach Round 3")
assign_round_probs("West Finals", "Reach Finals")
assign_round_probs("East Finals", "Reach Finals")
assign_round_probs("NBA Finals", "Win Championship")

# Optional: sort by title odds then finals odds
adv = adv.sort_values(["Win Championship", "Reach Finals"], ascending=False)

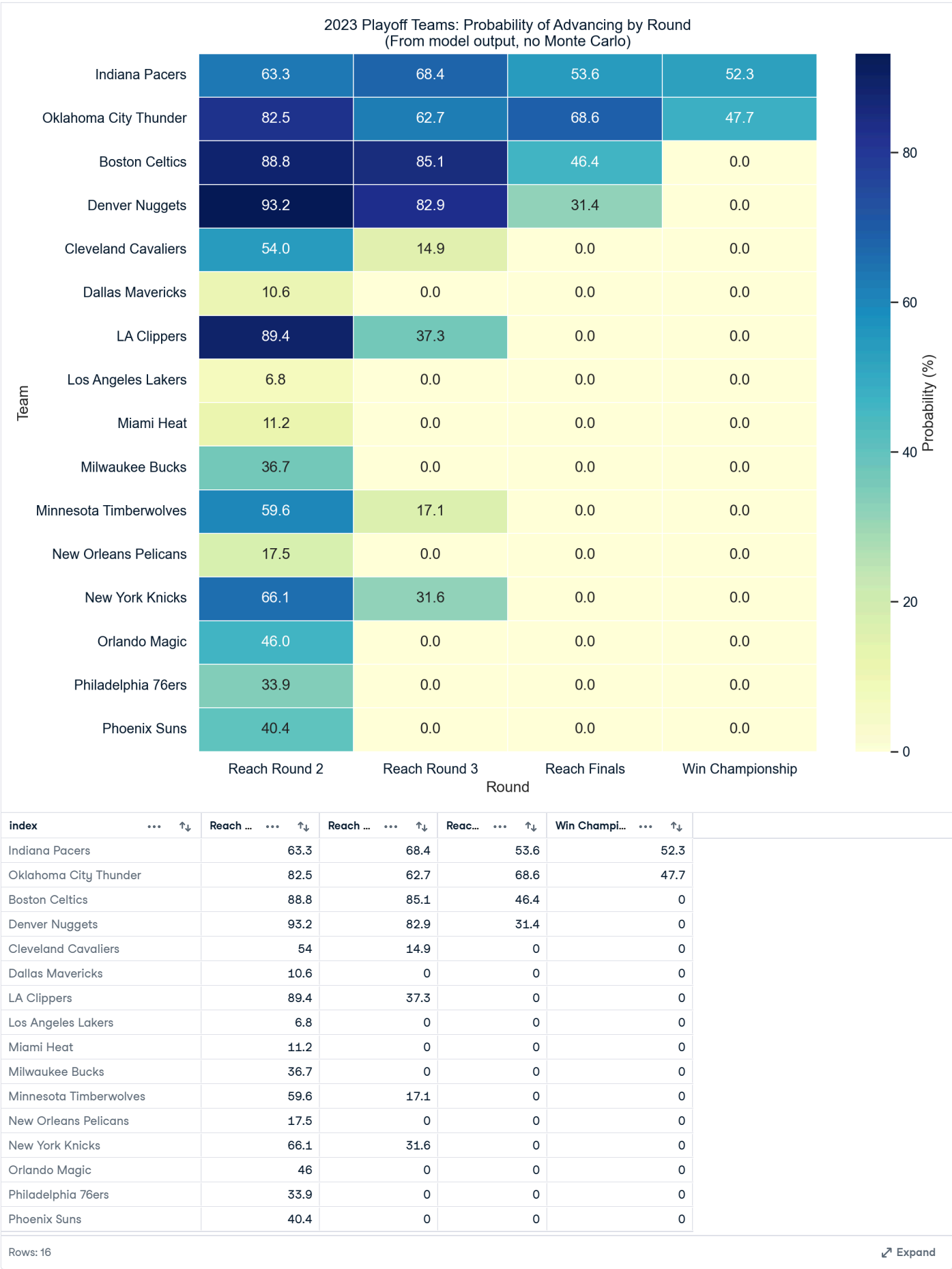
# 3) High-quality heatmap
plot_df = (adv * 100).round(1)

plt.figure(figsize=(11, 9))
sns.set_theme(style="whitegrid")
ax = sns.heatmap(
    plot_df,
    cmap="YlGnBu",
    annot=True,
    fmt=".1f",
    linewidths=0.6,
    cbar_kws={"label": "Probability (%)"})

ax.set_title("2023 Playoff Teams: Probability of Advancing by Round")
ax.set_xlabel("Round")
ax.set_ylabel("Team")
plt.tight_layout()
plt.show()

# 4) Also print a clean table for copy/paste
display(plot_df)

```



Discuss the strengths and weaknesses of your model as well as identify one team that did not win that was expected to win. Give one reason as bad luck and another as a cause not currently accounted for in your model. If given more time and data, how would you use what you found to improve your model?

The model works by first creating matchup based aligned metrics/statistics to each group of teams. The model trains on the previous 9 playoff seasons and uses each individual playoff game in its training. We first wanted to identify the 12 most important characteristics in our data of the 140 possible columns available (minus categorical identifiers). The model then creates a base model using an XGBClassifier and then returns it with the 12 most important metrics to go through again. If a team has a winning chance of greater than 50% they are considered to win the series. From the test, we then apply team _a metrics to the home team and team _b to the away team and simulate a playoff series 400 times. From there we inputted logic to assign each winning team chance of advancing and having home court advantage moving forward.

The model has an accuracy of 60.7% and a ROC of 61% suggesting the model performs better than random guessing.

Strengths 1--Team level features for both home and away teams so matchups are context specific than pulling an individual team's rankings 2--XGBoost model offers a better way to capture nonlinear interactions and handles contextual metrics better than linear models 3--By identifying a game-game home-away switch modeling, it offers a more dynamic predictability than team a versus team b without the context

Weaknesses 1-- For this level of effort, relatively small training/test split so potential for noisy outcomes 2--Randomized train/test split so overstating real-world 3--Aggregate specific; does not include trends, all team awards, injuries, etc. 4-No explicit calibration so model not perfectly calibrated and does not consider historical significance (ex. No team has ever come down 3-0)

Two teams that underperformed against expectation were the Milwaukee Bucks and the New York Knicks, who were both upsetted by the Indiana Pacers (ironically both in real life as well). A reason for this could be the pace at which the Pacers play (123ppg) is significantly greater than both the other 2 teams. They had significantly offensive stats than the Bucks and Knicks and this offense may have contributed to both teams underperforming that is not connected well. #If I had more time I would add greater trend information (last 10 games, last 20 games, etc) as well as better input from stars.